Practical - 6

Practical Title: Rendering External JSON Data using Fetch API, Search and Filter

Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

Theory :

1. How is JSON fetched, parsed, and rendered?

JSON data is fetched from an external JSON file using the Fetch API. The fetch() method returns a Promise, and response.json() parses the response into a JavaScript object or array. The parsed data is then processed and rendered dynamically into HTML elements using JavaScript, usually with map(), template literals, or DOM methods such as innerHTML.

2. Which array methods are used for search, filter, sort, and pagination?

Search can be performed using filter() together with includes() to find matching text. filter() is also used to select records according to a condition. sort() arranges records in ascending or descending order. For pagination, slice() is commonly used to select only the records belonging to the current page. map() can then be used to convert the selected records into HTML for rendering.

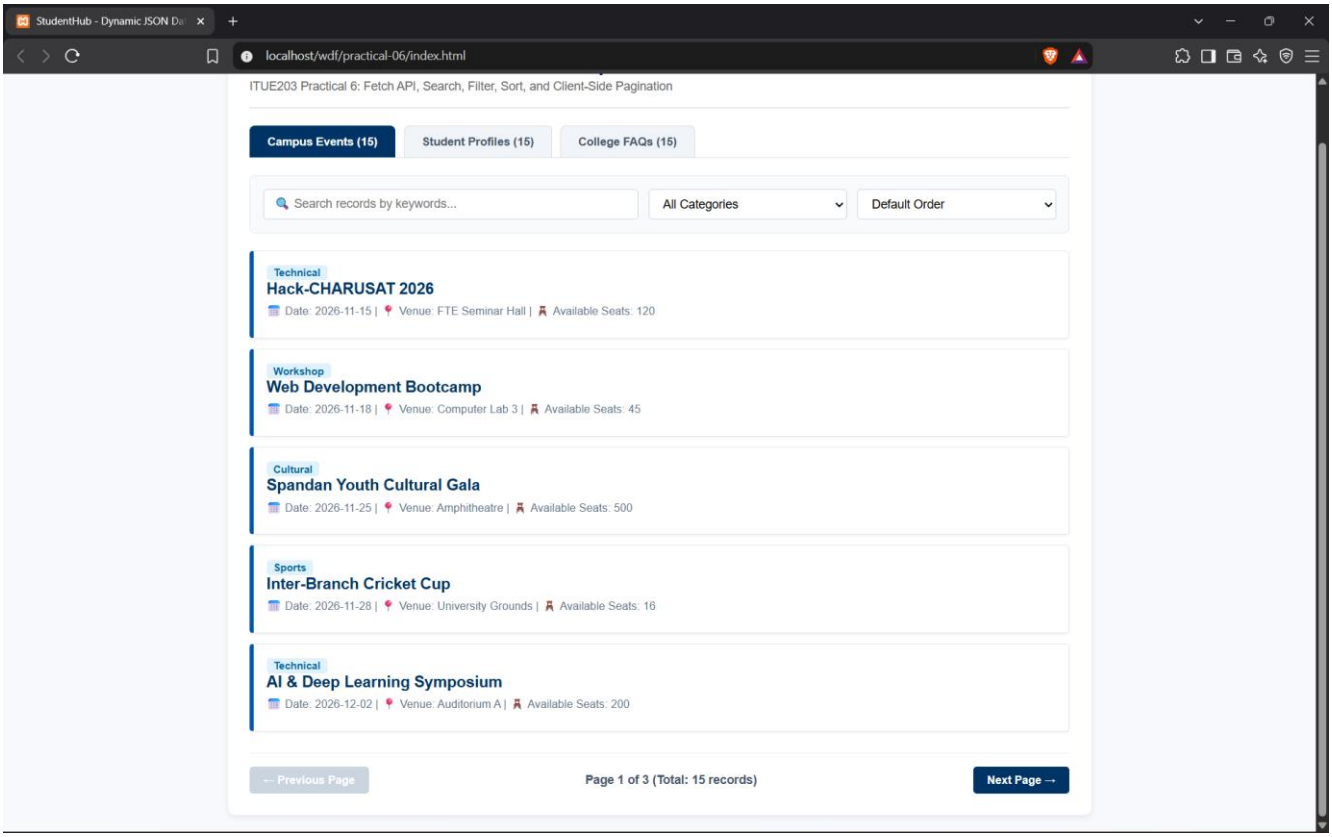
3. How are loading and error states handled?

A loading message is displayed before the JSON request is completed. After the fetch operation finishes, the loading message can be removed and the data can be displayed. If the request fails or the response is not successful, catch() or an equivalent error check displays a clear error message to the user instead of leaving the page blank.

4. How is modularity maintained in JavaScript files?

Modularity is maintained by separating responsibilities into functions or separate JavaScript modules. For example, one function can fetch JSON data, another can render records, and other functions can handle search, filtering, sorting, and pagination. ES6 import and export can be used when multiple JavaScript files are required. This makes the code easier to understand, test, reuse, and maintain.

Implementation/ Output/ Result


StudentHub - Dynamic JSON D
localhost/wdf/practical-06/index.html
ITUE203 Practical 6: Fetch API, Search, Filter, Sort, and Client-Side Pagination
Campus Events (15)
Student Profiles (15)
College FAQs (15)
Search records by keywords...
All Categories
Default Order
Technical
Hack-CHARUSAT 2026
Date: 2026-11-15 | Venue: FTE Seminar Hall | Available Seats: 120
Workshop
Web Development Bootcamp
Date: 2026-11-18 | Venue: Computer Lab 3 | Available Seats: 45
Cultural
Spandan Youth Cultural Gala
Date: 2026-11-25 | Venue: Amphitheatre | Available Seats: 500
Sports
Inter-Branch Cricket Cup
Date: 2026-11-28 | Venue: University Grounds | Available Seats: 16
Technical
AI & Deep Learning Symposium
Date: 2026-12-02 | Venue: Auditorium A | Available Seats: 200
← Previous Page
Page 1 of 3 (Total: 15 records)
Next Page →

Learning Outcomes:

- Understand and use JSON data in web applications.
- Fetch and parse external JSON data using the Fetch API.
- Render dynamic content using JavaScript and reusable frontend views.
- Implement search, filtering, sorting, and pagination using array methods.
- Handle loading and error states while fetching external data.
- Maintain modular JavaScript code for easier reuse and maintenance.

Dataset / Test Data:

- events.json – at least 15 event records.
- students.json – at least 15 student records.
- faqs.json – at least 15 FAQ records.

Tools / Technology Used:

- JavaScript ES6+
- JSON
- Fetch API
- HTML5
- CSS3

Post Laboratory Work:

Submit the JSON files, rendered views, and test screenshots demonstrating search, filtering, sorting, and pagination.

Advanced / Intermediate Extension:

- Intermediate: Add dependent dropdowns for country, state, and city.
- Advanced: Cache the latest JSON response in localStorage for offline-like display.

Viva Preparation:

- Explain how fetch() retrieves JSON data and how response.json() parses it.
- Explain the purpose of map(), filter(), sort(), and slice().
- Explain how search and filtering are applied to JSON records.
- Explain pagination and how the current page determines the displayed records.
- Explain how loading and error messages improve user experience.
- Explain how JavaScript modules and functions improve code organization.

GitHub Repository Link

https://github.com/dhyandesai1727/25DIT012-WDF-BATCH-A